

UNIVERSIDADE FEDERAL DE RORAIMA | DCC

Quick Sort

Da partição à engenharia de ordenação

Estrutura de Dados | Prof. Filipe Dwan Pereira

Nome: _____ Data: _____

Uma leitura algorítmica, histórica e experimental.

Roteiro e pergunta central

1. O que caracteriza o Quick Sort e como surgiu?
2. Como a partição produz subproblemas menores?
3. O que acontece, passo a passo, com seis valores?
4. Por que o desempenho médio é bom, mas o pior caso é quadrático?
5. O que a implementação e os dados da `lesson11` permitem concluir?

Ideia em uma frase

Escolher um pivô, particionar a sequência em torno dele e ordenar recursivamente os lados.

Objetivo: explicar a técnica com precisão e distinguir garantias teóricas de medições de uma execução.

O problema de ordenação

Definição

Dada uma sequência de chaves $A = (a_0, \ldots, a_{n-1})$, produzir uma permutação A' tal que

$$a'_0 \leq a'_1 \leq \cdots \leq a'_{n-1}.$$

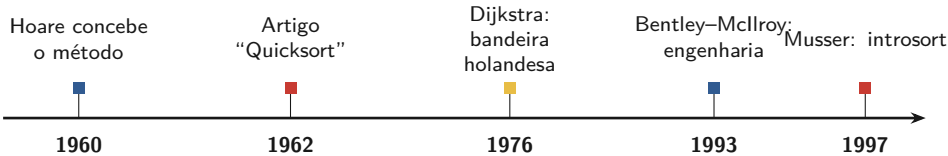
O conteúdo deve ser preservado; muda-se a ordem, não o multiconjunto de valores.

Propriedade de ordenação
Cada elemento aparece na posição compatível com a relação de ordem.

Propriedade de permutação
Nenhuma chave é perdida, criada ou duplicada durante o processo.

[2, 3]

Contexto histórico: uma ideia que virou família



- C. A. R. Hoare concebeu o algoritmo por volta de 1960; o artigo seminal saiu em 1962.
- O desenvolvimento posterior não é um único algoritmo: há diferentes partições, seleções de pivô e estratégias para casos especiais.

[1, 4, 5, 6, 7]

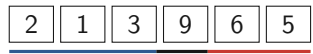
O que significa particionar?

Para um subvetor e pivô p, uma partição correta estabelece:

$$\underbrace{A[lo..q-1]}_{\leq p} \quad \underbrace{A[q]}_{p} \quad \underbrace{A[q+1..hi-1]}_{\geq p}$$

- O pivô termina em uma posição definitiva q.
- Os dois lados ainda podem estar desordenados.
- Ordenam-se os lados; não é necessário juntar (merge) depois.

[2, 4]



As relações exatas ($<$, $\leq$, etc.) dependem do esquema de partição e do tratamento de chaves iguais.

Dividir e conquistar: recorrência e custo

Se a partição deixa k elementos à esquerda:

$$T(n) = T(k) + T(n-k-1) + \Theta(n).$$

- Partições equilibradas: $T(n) = \Theta(n \log n)$.
- Partições extremas: $T(n) = \Theta(n^2)$.
- Para chaves distintas e pivô aleatório, o número esperado de comparações é $2(n+1)H_n - 4n$, aproximadamente $2n \ln n$.

[2, 4]

Leitura da recursão

Cada chamada paga pelo percurso da partição e depois resolve dois intervalos. O pivô é retirado dos subproblemas.

Espaço auxiliar: $O(1)$ para particionar in-place; além disso, a pilha recursiva é $O(\log n)$ em divisões equilibradas e $O(n)$ no pior caso.

Um detalhe importante: Hoare e Lomuto não são a mesma partição

	Esquema de Hoare	Esquema Lomuto-style
Movimento	Dois índices avançam de lados opostos.	Um índice de varredura e uma fronteira para os menores.
Pivô	Nem sempre retorna já na posição final.	O pivô final é trocado para a fronteira.
Recursão	Respeita a fronteira devolvida pela partição.	Exclui a posição final do pivô.
Observação	Pode fazer menos trocas em certos dados.	É simples de explicar, mas pode trocar muitas vezes e degradar com chaves iguais.

Na `lesson11`: o pivô é `values[right - 1]`; `store` avança para valores estritamente menores e, ao fim, recebe o pivô. É uma partição do tipo Lomuto.

[2, 4]

Passo a passo (1/2): seis números, pivô 3

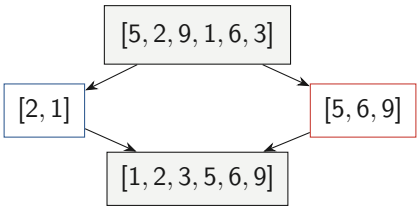
Entrada: [5, 2, 9, 1, 6, 3]. Índices começam em zero; pivô é o último item.

j	Valor	Teste	Vetor após o passo
–	–	Estado inicial; `store`=0	$[5, 2, 9, 1, 6, 3]$
0	5	$5 < 3$? não	$[5, 2, 9, 1, 6, 3]$
1	2	$2 < 3$? sim; troca $A[0] \leftrightarrow A[1]$	$[2, 5, 9, 1, 6, 3]$
2	9	$9 < 3$? não	$[2, 5, 9, 1, 6, 3]$
3	1	$1 < 3$? sim; troca $A[1] \leftrightarrow A[3]$	$[2, 1, 9, 5, 6, 3]$
4	6	$6 < 3$? não	$[2, 1, 9, 5, 6, 3]$

`store`=2; valores menores que 3 estão à esquerda da fronteira. A varredura terminou.

Passo a passo (2/2): fixar o pivô e recursar

1. Trocar $A[\texttt{store}]$ com o pivô: $[2, 1, 3, 5, 6, 9]$. O 3 está na posição final 2.
2. Recursar à esquerda: $[2, 1]$, pivô $1 \rightarrow [1, 2]$.
3. Recursar à direita: $[5, 6, 9]$, pivô $9 \rightarrow [5, 6, 9]$.
4. Juntar não é necessário: a concatenação dos intervalos e do pivô já dá a ordem.



Esta execução ilustra o rastreamento do código usado na implementação, não todas as variantes de Quick Sort.

Pseudocódigo: partição Lomuto e Quick Sort

Particionar o intervalo $[lo, hi)$

1. $p \leftarrow A[hi-1]$; $s \leftarrow lo$
2. Para $j = lo, \ldots, hi-2$:
   - se $A[j] < p$, trocar $A[j]$ com $A[s]$; $s \leftarrow s+1$
3. Trocar $A[s]$ com $A[hi-1]$
4. Devolver s

Ordenar

1. Se $hi - lo < 2$, retornar.
2. $q \leftarrow \text{Particionar}(A, lo, hi)$
3. Ordenar $A[lo..q)$
4. Ordenar $A[q+1..hi)$

Invariante

Durante a varredura, $A[lo..s-1] < p$; os elementos ainda não examinados ficam à direita do índice j.

[2]

C: implementação da partição

```c
static size_t partition(int *a, size_t lo, size_t hi)
{
    int pivot = a[hi - 1];
    size_t store = lo;

    for (size_t j = lo; j + 1 < hi; ++j) {
        if (a[j] < pivot) {
            int tmp = a[j];
            a[j] = a[store];
            a[store] = tmp;
            ++store;
        }
    }

    int tmp = a[store];
    a[store] = a[hi - 1];
    a[hi - 1] = tmp;
    return store;
}
```

Pivô no último índice do intervalo semiaberto $[lo, hi)$; a partição devolve sua posição definitiva.

C: recursão sobre os intervalos

```c
static void quick_sort(int *a, size_t lo, size_t hi)
{
    if (hi - lo < 2) return;
    size_t q = partition(a, lo, hi);
    quick_sort(a, lo, q);
    quick_sort(a, q + 1, hi);
}
```

A chamada inicial para n elementos é `quick_sort(a, 0, n)`. O código da `lesson11` acrescenta métricas e atualização de progresso.

Propriedades: o que o Quick Sort oferece (e não oferece)

Pontos fortes

- Bom desempenho médio: $\Theta(n \log n)$.
- Particionamento in-place, com boa localidade de acesso.
- Laço interno compacto; baixo custo auxiliar fora da pilha.
- Pode ser adaptado a distintos tipos de dados e padrões.

Limitações

- Pior caso $\Theta(n^2)$ com partições repetidamente desequilibradas.
- Versão recursiva pode usar $O(n)$ de pilha.
- Não é estável em sua forma usual.
- Resultado depende da política de pivô, da partição e de chaves repetidas.

"In-place" não significa memória total $O(1)$: a recursão também ocupa espaço.

[2, 3, 4]

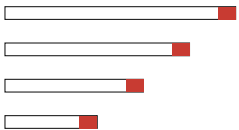
Quando o pivô dá errado?

Considere $1, 2, 3, 4, 5$ com o maior elemento sempre escolhido como pivô:

$$4 + 3 + 2 + 1 = \frac{n(n-1)}{2}$$

comparações no caso extremo.

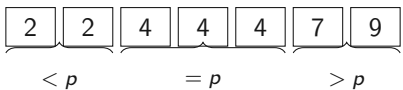
- Pivô fixo pode ser vulnerável a entradas já ordenadas ou construídas para a implementação.
- Entradas com muitas chaves iguais exigem tratamento consciente.



[2, 9]

Chaves repetidas: partição em três vias

Separar em três regiões: $< p$, $= p$ e $> p$. A região igual não precisa ser recursada.



- A solução de Dijkstra para a "bandeira holandesa" mantém fronteiras para as três regiões.
- Em dados com muitas duplicatas, essa técnica evita recursão inútil sobre valores iguais; Bentley–McIlroy a incorporaram a uma implementação prática de ordenação.
- A implementação da `lesson11` não é tri-partida: usa comparação estrita `< pivot`.

[5, 6, 4]

Melhorias modernas: Quick Sort como componente

Técnica	O que busca resolver
Pivô aleatório	Reduz a dependência de uma entrada adversa fixa; expectativa $O(n \log n)$.
Mediana de três / amostragem	Evita pivôs extremos em padrões comuns, sem garantir o pior caso.
Partição em três vias	Trata chaves repetidas com uma única região de igualdade.
Inserção em intervalos pequenos	Diminui custos constantes e chamadas recursivas curtas.
Introsort	Monitora a profundidade e recorre ao Heapsort para limitar o pior caso a $O(n \log n)$.
Dual-pivot / variantes adaptativas	Exploram políticas diferentes de partição e padrões dos dados; desempenho depende da implementação.

Exemplo de uso em biblioteca: a documentação de Java descreve Dual-Pivot Quicksort para certas ordenações de arrays primitivos. A estratégia concreta de `std::sort` depende da biblioteca; não se deve inferir uma implementação a partir apenas do nome da API. [7, 6, 10]

Comparação com outros algoritmos do trabalho

Algoritmo	Tempo típico / pior	Estável?	In-place?	Observação principal
Quick Sort	$n \log n$ / n^2	Não	Em geral	Rápido na prática; o pivô importa.
Merge Sort	$n \log n$ / $n \log n$	Sim*	Não*	Garantia de pior caso; memória auxiliar.
Heap Sort	$n \log n$ / $n \log n$	Não	Sim	Pior caso garantido; acesso menos sequencial.
Insertion Sort	n^2 / n^2	Sim	Sim	Simples e eficiente para entradas pequenas/quase ordenadas.
Selection Sort	n^2 / n^2	Não*	Sim	Poucas trocas; muitas comparações.
Bubble Sort	n^2 / n^2	Sim*	Sim	Didático; depende da otimização para parar cedo.
Radix Sort	$d(n+b)$ / $d(n+b)$	Depende	Não*	Não compara chaves; custo depende da representação e base.

$*$ Propriedades da versão usual/implementação estável, não de todo algoritmo que compartilha o nome. d: número de dígitos/passagens; b: base ou tamanho do alfabeto.

[2, 8, 3]

Busca: o que muda quando os dados estão ordenados?

Busca sequencial

Examina itens até encontrar a chave ou chegar ao fim.

Pior caso: $O(n)$. Não exige ordenação.

Busca binária

Compara com o elemento central e descarta metade do intervalo a cada etapa.

Pior caso: $O(\log n)$. Exige sequência ordenada e acesso indexado eficiente.

Na `lesson11`, o benchmark de buscas usa 10.000 chaves e 10.000 consultas pseudoaleatórias: sequencial no vetor original e binária em uma cópia ordenada.

[2]

Metodologia experimental da `lesson11`

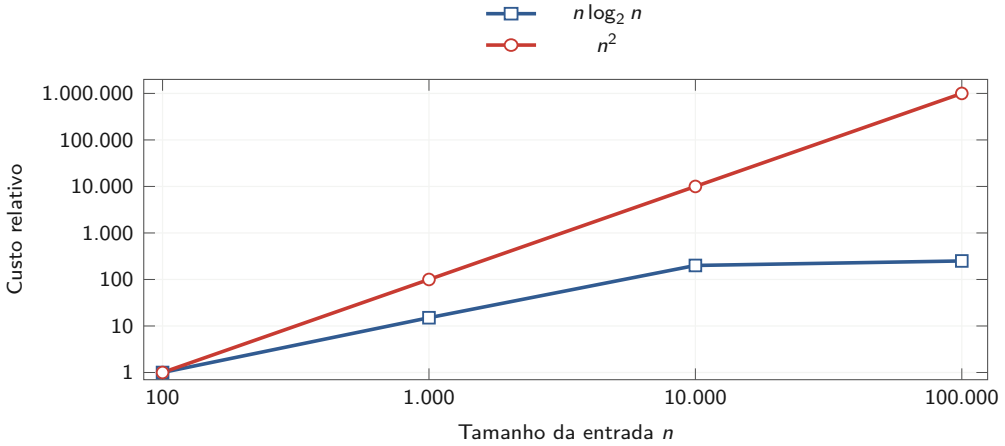
- O módulo executa cada algoritmo sobre arquivos de entrada de 100, 1.000, 10.000 e 100.000 inteiros.
- Mede o intervalo da chamada ao algoritmo com contador de alta resolução do Windows; o tempo de gravação do arquivo é posterior à medição.
- Os arquivos ordenados preservam os valores da entrada. A checagem local dos 21 resultados disponíveis confirmou ordem não decrescente e mesmo multiconjunto.
- Os nomes dos arquivos guardam o tempo arredondado para microssegundos inteiros ou milissegundos inteiros; os arquivos não registram máquina, compilador, repetições ou dispersão.

Escopo observado

Há artefatos de dados para sete ordenações até 10.000 elementos. Para 100.000, há resultados de Heap, Merge, Quick e Radix; não há arquivos dessa dimensão para Bubble, Insertion e Selection.

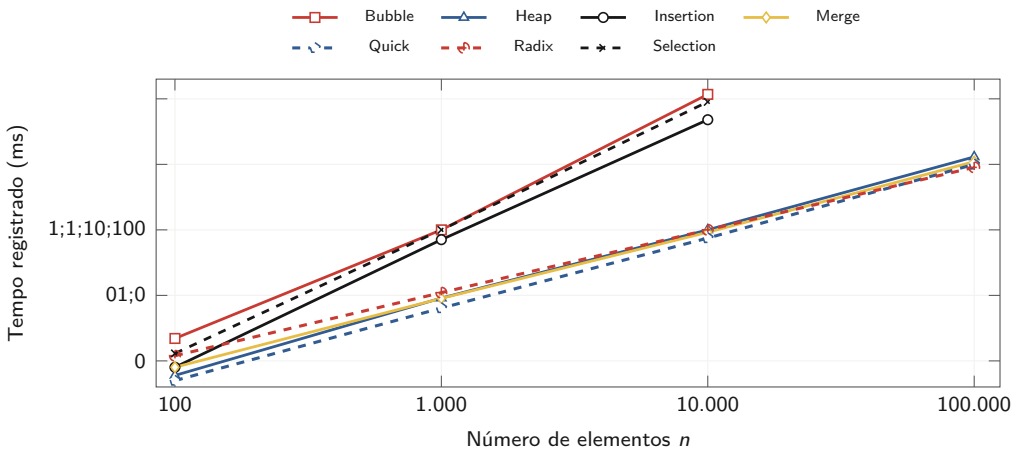
Fonte: `problems/lesson11/problem_110.c`, `search-benchmark.c` e `data/`.

Expectativa teórica: crescimento, não previsão de milissegundos



Curvas normalizadas em $n = 100$. A escala log–log evidencia a diferença entre crescimento linearítmico e quadrático; não são tempos medidos.

Resultados práticos registrados: crescimento por tamanho



Escala log–log para tornar comparáveis 0,005 ms e 117 ms. Pontos de 100.000 existem somente para Heap, Merge, Quick e Radix; as linhas ausentes não foram extrapoladas.

Curvas teóricas versus medidas: escala relativa

Previsão assintótica normalizada

Modelo	100	1.000	10.000	100.000
$n \log_2 n$	1	15,1	200,1	2.501
n^2	1	100	10.000	1.000.000
n	1	10	100	1.000

A normalização elimina a constante desconhecida e permite comparar a forma do crescimento.

Tempos medidos normalizados

Algoritmo	100	1.000	10.000	100.000
Quick	1	12,8	150	2.000
Heap	1	15,2	166,7	2.166,7
Merge	1	11,3	115	1.375
Radix	1	9,2	83,3	750

Base: o menor arquivo de cada algoritmo; valores aproximados pelos nomes dos arquivos.

Leitura: Quick, Heap e Merge crescem na mesma ordem de grandeza esperada para $n \log n$, embora suas constantes e arredondamentos sejam diferentes. Radix apresenta crescimento próximo de linear nesta faixa, coerente com d e a base fixados, mas não é diretamente comparável a um algoritmo baseado em comparações.

Comparação por algoritmo: previsão e evidência

Algoritmo	Previsão teórica	Curva observada	Conclusão para esta execução
Quick Sort	Médio $n \log n$; pior n^2	$0,005 \rightarrow 10$ ms; fator ≈ 2.000	A série observada acompanha a família $n \log n$; pivô final e entrada aleatória não produziram degradação quadrática.
Heap Sort	$n \log n$ em todos os casos	$0,006 \rightarrow 13$ ms; fator ≈ 2.167	Crescimento próximo ao de Quick/Merge; oferece garantia de pior caso.
Merge Sort	$n \log n$ em todos os casos	$0,008 \rightarrow 11$ ms; fator ≈ 1.375	Compatível com $n \log n$, com constante favorável nesta medição.
Radix Sort	$d(n + b)$; linear se d, b fixos	$0,012 \rightarrow 9$ ms; fator ≈ 750	Crescimento mais próximo de n na faixa; depende do domínio numérico e da implementação.
Insertion Sort	n^2 no pior/médio caso	$0,008 \rightarrow 48$ ms em 10 mil; 100 mil ausente	A forte subida entre 1 mil e 10 mil é compatível com custo quadrático; não extrapolar como medição.
Selection Sort	n^2	$0,013 \rightarrow 79$ ms em 10 mil; 100 mil ausente	Tendência quadrática aparente; a ausência de 100 mil impede afirmar o tempo real.
Bubble Sort	n^2 (versão usual)	$0,022 \rightarrow 117$ ms em 10 mil; 100 mil ausente	Maior custo entre os quadráticos medidos; não há curva observada em 100 mil.

"Compatível" significa que os pontos não contradizem a ordem de crescimento esperada; não é uma prova estatística da complexidade.

Como interpretar as curvas sem confundir teoria e prática

1. **Teoria:** compara ordens de crescimento quando n aumenta; ignora constantes, hardware, cache e implementação.
2. **Prática:** os nomes dos arquivos registram uma execução arredondada; pontos próximos podem parecer iguais por causa da resolução.
3. **Curvas paralelas:** Quick, Heap e Merge mantêm crescimento semelhante no gráfico de 100 a 100.000.
4. **Curvas ausentes:** não há ponto de 100.000 para Bubble, Insertion e Selection; uma linha não deve ser inventada por extrapolação.
5. **Radix:** sua previsão depende de d e b; a comparação com $n \log n$ é uma comparação de modelos, não de operações equivalentes.

Mensagem para a banca

Os dados são coerentes com as previsões assintóticas nas entradas observadas, mas a conclusão deve ser limitada ao ambiente, às entradas e aos arquivos efetivamente medidos.

Leitura dos resultados: o que os dados sustentam

Algoritmo	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$	Leitura cuidadosa
Quick Sort	5 μs	64 μs	750 μs	10 ms	Menor tempo registrado nos quatro tamanhos disponíveis.
Heap Sort	6 μs	91 μs	1 ms	13 ms	Crescimento compatível com $n \log n$.
Merge Sort	8 μs	90 μs	920 μs	11 ms	Mesmo patamar assintótico de Quick e Heap.
Insertion Sort	8 μs	709 μs	48 ms	–	Não há arquivo de 100 mil.
Selection Sort	13 μs	1 ms	79 ms	–	Não há arquivo de 100 mil; custo quadrático sugere crescimento elevado.
Bubble Sort	22 μs	1 ms	117 ms	–	Não há arquivo de 100 mil; custo quadrático sugere crescimento elevado.
Radix Sort	12 μs	110 μs	1 ms	9 ms	Melhor resultado registrado em 100 mil nesta amostra.

Nomes consultados: `100000_13ms.txt`, `100000_11ms.txt`, `100000_10ms.txt` e `100000_9ms.txt`; tempos arredondados pela rotina de gravação.

O que não se pode concluir apenas com esses tempos

- Uma execução por algoritmo/tamanho não estima variabilidade nem intervalo de confiança.
- Resolução diferente entre microssegundos e milissegundos esconde diferenças em alguns pontos.
- Não foram registrados aqui CPU, sistema, compilador, flags, estado de cache ou distribuição detalhada das entradas.
- Quick Sort é o menor tempo *registrado* nesta amostra, mas isso não prova superioridade universal.

Como fortalecer o experimento

Executar várias repetições; usar as mesmas entradas e cópias independentes; relatar mediana e dispersão; registrar compilador/flags e hardware; testar aleatória, ordenada, reversa e muitos valores iguais; completar os três algoritmos quadráticos em 100.000 somente se o custo for aceitável.

Por que ainda escolher Quick Sort?

É uma boa escolha quando

- dados cabem em memória;
- estabilidade não é requisito;
- desempenho médio e localidade são prioritários;
- a implementação pode escolher boas políticas de pivô e fallback.

Considere alternativas quando

- pior caso garantido é obrigatório (Heap/Merge/Introsort);
- estabilidade é obrigatória (Merge);
- dados são quase ordenados e pequenos (Insertion);
- a chave tem estrutura explorável (Radix).

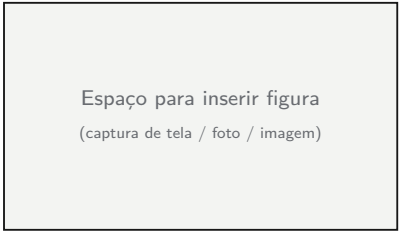
A escolha é uma decisão de engenharia: modelo de dados, requisitos e implementação importam tanto quanto a notação assintótica.

[3, 2, 8]

Conexão com o trabalho da disciplina

- A implementação integra o menu de comparação da `lesson11`.
- A rotina acompanha comparações, movimentos e progresso.
- Arquivos ordenados são gravados em `data/quick-sort/`.
- Os artefatos disponíveis cobrem 100, 1.000, 10.000 e, parcialmente, 100.000 números; foram validados quanto à ordenação e preservação dos valores.

Fontes: `problems/lesson11/quick-sort.c`, `problem_110.c` e `data/quick-sort/`.



Substituir por uma captura da tela de comparação ou do menu da `lesson11`.

Síntese

1. Quick Sort ordena por partições sucessivas e é um exemplo clássico de dividir e conquistar.
2. A qualidade do particionamento determina a forma da recursão: $n \log n$ quando equilibrada; n^2 no pior caso.
3. A implementação da `lesson11` usa pivô final e partição Lomuto-style; duplicatas e pivôs extremos são pontos de atenção.
4. Variantes modernas combinam partições melhores com inserção, três vias ou fallback para limitar riscos.
5. Os resultados disponíveis favorecem Quick Sort nesta amostra, mas a conclusão deve respeitar resolução e limites experimentais.

Obrigado(a)!

Perguntas?

Referências I


HOARE, C. A. R. Quicksort. *The Computer Journal*, v. 5, n. 1, p. 10–16, 1962. doi:10.1093/comjnl/5.1.10.


CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.


KNUTH, D. E. *The Art of Computer Programming, Volume 3: Sorting and Searching*. 2. ed. Addison-Wesley, 1998.


SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011. Material de apoio: `https://algs4.cs.princeton.edu/23quicksort/`.


DIJKSTRA, E. W. *A Discipline of Programming*. Prentice-Hall, 1976.


BENTLEY, J. L.; McILROY, M. D. Engineering a sort function. *Software: Practice and Experience*, v. 23, n. 11, p. 1249–1265, 1993. doi:10.1002/spe.4380231105.

Referências II


MUSSER, D. R. Introspective sorting and selection algorithms. *Software: Practice and Experience*, v. 27, n. 8, p. 983–993, 1997. doi:10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-#.


ZIVIANI, N. *Projeto de Algoritmos: com implementações em Pascal e C*. 3. ed. Cengage Learning, 2011.


MARTINEZ, C. Quicksort. In: BLACK, P. E. (ed.). *Dictionary of Algorithms and Data Structures*. NIST, 2023. Disponível em: `https://xlinux.nist.gov/dads/HTML/quicksort.html`.


ORACLE. *Java Platform, Standard Edition 8 API Specification: Arrays*. Nota de implementação sobre Dual-Pivot Quicksort. `https://docs.oracle.com/javase/8/docs/api/java/util/Arrays.html`.


CARAUMA, J. A. Repositório *art-mondrian*, módulo `problems/lesson11`. Código e dados experimentais da implementação. Disponível em: `https://github.com/carauma7/art-mondrian`.